Contents

Connect Four — Implementation Plan

> **Status:** Draft, 2026-05-17. Supersedes `Platform_Implementation_Plan.md` §Phase 4 — the original "drop in another game package" framing pre-dated the 2026 design lock that elevated Connect 4 into a multi-phase architectural initiative.
>
> **Related docs:** - `Game_SDK_Developer_Guide.md` — contract every game package must conform to - `Platform_Implementation_Plan.md` — broader platform roadmap; Phase 3.8 (multi-skill bots) is the prerequisite and is shipped - `Help_Corpus/matches-and-games.md`, `match-formats.md`, `match-design-rationale.md`, `solved-games-and-master.md` — match semantics + Master tier (already published to corpus) - `Help_Corpus/algorithm-history-*.md`, `the-history-of-game-ai.md` — algorithm background pack (already published to corpus)

Goal

Ship Connect 4 as the platform's second game, alongside Tic-Tac-Toe, with:

- Strict SDK conformance (game logic lives entirely in `packages/game-connect-four/`; if the SDK is deficient, extend the SDK rather than smuggling game logic into the platform).
- Match-based play with alternating colors and match-level ELO (best-of-2 ranked, best-of-3 tournament, casual stays best-of-1).
- All five training algorithms (Rule-Based, Minimax, MCTS, DQN, AlphaZero) operating through a unified async training architecture with realtime W/D/L streaming, multi-curve eval, preset gating, checkpoint resume, and a dedicated worker process.
- A Master tier (perfect-play minimax solver, off-ladder) for the AlphaZero-teaching moment.
- Mobile column-input UI (tap-to-preview, tap-to-confirm).

Design principle: architecture before game

The first half of the plan touches **no Connect 4 code at all**. Every architectural change required by Connect 4 (slug rename, game-as-prefix routing, match-based play, training rework, worker process, multi-skill bot UI) is implemented and validated against the existing Tic-Tac-Toe game first.

Rationale: each architectural change is a potential regression vector. Doing them under the cover of "we're shipping a new game" tangles new-feature risk with refactor risk. Doing them on TTT first lets us regress-detect against a familiar surface, with no rollback ambiguity. Only after the platform reaches architectural parity does Connect 4 code begin.

This is enforced in the phase structure below: **Phase A is TTT-only**, **Phase B is the C4 build**, **Phase C is platform polish**.

Phase A — Architectural alignment (TTT only, no C4 code)

> **Five sprints.** Each ships through the normal `dev → staging → main` deploy flow. End state: TTT works identically to today from a user's perspective, but the platform underneath is ready to host a second game.

A1 — SDK audit + game-as-prefix routing + slug rename

Goal: Move TTT onto the routes and slug that the platform will use long-term, with the SDK contract extended to cover anything Connect 4 will need.

- ☐ SDK audit — confirm the current contract (per `Game_SDK_Developer_Guide.md`) suffices for a 6×7 column-input game with a perfect-play "Master" tier. Identify any gaps.
- ☐ Extend SDK if needed (likely: `meta.inputMode: 'cell' | 'column'`, `meta.matchFormat` hints, `meta.masterStrategy` hook for off-ladder perfect-play bots). Bump SDK version; update `Game_SDK_Developer_Guide.md`.
- ☐ Refactor `packages/game-xo/` to consume any new SDK fields explicitly. No hidden TTT assumptions.
- ☐ Add `/games/<slug>/...` route layer to backend (`/api/v1/games/<slug>/...`), landing (`/games/<slug>/...`), and tournament service.
- ☐ DB migration: rename `xo` → `tic-tac-toe` in `BotSkill.gameId`, `GameElo.gameId`, `Tournament.game`, and any other gameId-bearing rows. Single migration; no dual-write phase.
- ☐ Backend code: strip `GAME_ID = 'xo'` constant from `eloService.js`; parameterize every gameId-aware function.
- ☐ Backend cache keys: rename `bots:gameId:xo` → `bots:gameId:tic-tac-toe`. Flush on deploy.
- ☐ Landing: update `BotFilterBar.GAMES`, route definitions, deep-link parsers, picker UI.
- ☐ 301 redirects from old paths (`/play/xo` → `/games/tic-tac-toe/play`, etc.) for one release; remove in A2 cleanup.
- ☐ Update all docs in `/doc/Help_Corpus/` that reference the slug `xo` (search-and-replace pass).
- ☐ Tests: regression suite passes; e2e journey + smoke pass on staging.

A2 — Match-based play + match-level ELO (TTT)

Goal: Ranked TTT becomes best-of-2 with alternating colors. Tournament becomes best-of-3 with random-color game-3 tiebreaker. ELO updates per match, not per game. Casual stays best-of-1.

- ☐ New `Match` entity in DB (or extend `Game` with `matchId`, `matchSequence`). Decide schema: pure relational `Match` row with child `Game` rows, or denormalized `matchId` column on `Game`.
- ☐ Match lifecycle states: `FORMING → IN_PROGRESS → COMPLETED`, with per-game results aggregated to a match score.
- ☐ Color assignment logic — game 1 random/seeded, subsequent games swap, game 3 tournament tiebreaker random with announcement.
- ☐ Ranked play: server orchestrates a best-of-2 match for any "Ranked vs X" entry point. No client-side color choice.
- ☐ Tournament play: extend existing `TournamentMatch` (`p1Wins`, `p2Wins`, `drawGames` already exist) to enforce best-of-3 with the random-color rule for game 3.
- ☐ `eloService.js`: implement match-score ELO update (sum of per-game 1.0/0.5/0.0 scores ÷ N games), replacing the per-game update for ranked/tournament. Casual single-game update unchanged.
- ☐ Historical rows: stay in place. No retroactive recompute. Pre-cutover ELO is the starting point; new matches move from there.
- ☐ Realtime: emit match-level events (`match.started`, `match.gameComplete`, `match.completed`) on the existing realtime channel. Client renders match progress (1-0, 1-1, etc.).
- ☐ Landing UI: ranked entry points say "Best of 2" explicitly. Match progress visible during play.
- ☐ Tests: ELO math worked-example test (matches the example in `Help_Corpus/match-formats.md`). E2E ranked-match flow. Tournament best-of-3 with 1-1 tiebreaker.
- ☐ Corpus alignment: `matches-and-games.md`, `match-formats.md`, `match-design-rationale.md` are already published — confirm they describe the shipped behavior.

A3a — Training data + UX rework (in-process, TTT)

Goal: Reshape the training API into the form Connect 4 will need, without yet moving training out of the backend process. TTT validates everything.

- ☐ New tables: `TrainingSession` (or extend existing — current `TrainingSession` is close; audit gaps) and `TrainingMetric` (time-series rows for W/D/L curve points + checkpoints).
- ☐ Channel rename: realtime training updates move from `ml:session:{id}` to `training:<sessionId>`.
- ☐ Preset model: Quick / Standard / Deep presets per (algorithm, game) with explicit episode count, expected duration, and ETA estimate computed up front.
- ☐ ETA-threshold gating — sessions ≥4 hours require admin approval. Sub-4hr sessions auto-confirm. Approval flow has a queue UI for admins.
- ☐ Per-user concurrency cap: 1 active training session per user (configurable per role).

- ☐ Multi-curve eval: each session periodically plays N games against the algorithm-appropriate primary opponent + Easy + Medium side curves. Budget-split sampling (~12 primary + 4 Easy + 4 Medium, total ~20 games per eval), keeping overhead ~5%.
- ☐ Stacked W/D/L visualization: each eval point emits W/D/L tallies. Frontend renders stacked area chart per opponent curve. Saturated curves (e.g., 100% wins against Easy) auto-fade.
- ☐ Master-vs-bot split (deferred wiring; lights up in Phase B): if eval includes Master, split into "as first-mover" and "as second-mover" sub-charts. No-op for TTT (no Master tier).
- ☐ Checkpoint-based recovery: every N episodes, persist a checkpoint. On process crash, session resumes from the last checkpoint on next backend boot.
- ☐ Resume button: users can pause and resume sessions (not yet "stop early"). Stop-early is a Phase C item.
- ☐ No knobs exposed in the v1 UI — presets only. Advanced disclosure deferred.
- ☐ Tests: TrainingSession + TrainingMetric CRUD, preset selection, ETA gating boundary at 4hr, multi-curve eval budget math, checkpoint resume after simulated crash.
- ☐ Corpus: write the queued training UX docs (how sessions work, reading W/D/L charts, reading multi-curve view, when to stop, time/cost, resuming, self-play vs eval).

A3b — Training worker process cutover (TTT)

Goal: Move training out of the backend process into a dedicated worker, so a long AlphaZero Deep run can never starve the API box.

- ☐ New Fly app: `xo-training-prod` (and `-staging`), sharing the backend container image. Separate scale + resources from the API backend.
- ☐ Redis-backed job queue (BullMQ or equivalent; we already have Redis in the stack). Backend enqueues `startTraining` jobs; worker(s) consume.
- ☐ Pub/sub channel: worker emits training events to Redis; backend SSE handler subscribes and fans out to connected clients. Realtime contract from A3a is unchanged (clients still consume `training:<sessionId>` via the backend).
- ☐ Graceful shutdown: worker handles `SIGTERM` by checkpointing the current session, persisting state, and exiting cleanly. New worker on next deploy resumes from checkpoint.
- ☐ Cap enforcement at the queue: per-user cap (1 active) + global cap (configurable, e.g., 4 concurrent sessions) gated before a job leaves the queue.
- ☐ Failure handling: job retries on transient errors, dead-letter after N failures. Sessions in dead-letter surface to admin with a "diagnostic dump" link.
- ☐ Observability: per-worker CPU/RAM dashboard, queue depth alert, dead-letter alert. Hook into `Observability_Plan.md`.
- ☐ Tests: queue enqueue/consume happy path, cap enforcement, graceful-shutdown checkpoint, crash-during-training recovery, dead-letter flow.
- ☐ Deploy: `fly launch` for `xo-training-staging`, wire to staging Redis/DB. Validate TTT training under load (multiple concurrent users, mix of algorithms). Promote to prod when green.

A4 — Multi-skill bot UI + auto-clone

Goal: Finish the Phase 3.8 remainder. By the time Connect 4 ships, multi-game bot management is already in users' hands.

- ☐ Bot detail page: per-skill cards listing every `BotSkill` row owned by the bot, with per-game ELO and last-trained timestamps.
- ☐ Add-skill flow: from a bot, pick a game the bot doesn't have a skill for, pick an algorithm, optionally clone hyperparams from an existing skill. Wire to `repointBotPrimarySkill` so `User.botModelId` updates correctly.
- ☐ Auto-clone non-primary skills on rename/rebrand: when the bot is renamed, every skill carries along (already true; verify and test).
- ☐ Picker disambiguation: when a bot has multiple skills, the picker shows "BotName (game)" with the active gameId, so users always know what they're queuing.
- ☐ Profile + Gym nav: deep-link straight to a skill's training page from the bot card.
- ☐ Pre-C4 validation: with only TTT live, the UI is intentionally lightly populated. We're validating the data plumbing, not showcasing breadth.
- ☐ Tests: add-skill happy path, rename across multiple skills, picker disambiguation, primary-skill repoint after training completes.

Phase A exit criteria

- ☐ TTT plays identically end-to-end (casual + ranked + tournament) on the new routes and new slug.
- ☐ TTT match-level ELO produces sensible movements for the existing user base.
- ☐ All 5 TTT algorithms train through the worker process with W/D/L streaming, multi-curve eval, and checkpoint resume.
- ☐ Multi-skill bot UI lets a user add, manage, and queue from multiple skills (limited to one game today).
- ☐ Help corpus updated: any TTT-facing docs that reference the old slug, single-game ranked, or the old training UX are refreshed.
- ☐ No open regressions on the V1 acceptance script.

Phase B — Connect 4 implementation

> **Five sprints.** Each builds atop the architecture established in Phase A. If any sprint here surfaces an SDK gap, the work pauses, the SDK is extended (with `Game_SDK_Developer_Guide.md` updated), and the sprint resumes — game logic never leaks into the platform.

B1 — `packages/game-connect-four/` package

Goal: A standalone, SDK-conformant game package. Engine + bot interface + tests, no UI yet.

- ☐ Package scaffold: `packages/game-connect-four/` with the same shape as `packages/game-xo/`.
- ☐ Engine: 6×7 board, gravity mechanic, win detection (horizontal/vertical/both diagonals), draw detection (board-full), legal-move enumeration (top-of-column).
- ☐ State serialization: compact format suitable for replay storage and bot weight inputs.
- ☐ `meta` export: `id: 'connect-four'`, `inputMode: 'column'`, `matchFormat: { ranked: 'bo2', tournament: 'bo3', master: 'bo2' }`, `masterStrategy: 'solver'`, `builtInBots: [easy, medium, hard, master]`.
- ☐ `botInterface`:
  - Built-in minimax tiers (Easy depth-2, Medium depth-4, Hard depth-6 to depth-8, Master = the solver).
  - Training hooks for the five algorithms (delegating to the shared `@xo-arena/ai` engines where game-agnostic; new code only where C4 specifics matter).
- ☐ Master solver: perfect-play minimax with full-depth search and the known solved-game heuristics (center-first opening, threat parity). Off-ladder.
- ☐ Unit tests: engine correctness (win/draw/illegal-move), serialization round-trip, minimax depth correctness, solver vs known opening lines.

B2 — C4 play surface (desktop + mobile)

Goal: Casual quick match against built-in Easy/Medium/Hard works end-to-end via `/games/connect-four/...`.

- ☐ Game component: 6×7 board with Red/Yellow tokens, drop animation, win highlight, draw indicator.
- ☐ Desktop input: click on column to drop.
- ☐ Mobile input: tap-to-preview (semi-transparent token at the top of the column showing where the piece will land), tap-to-confirm (drops the piece). Tap a different column to switch preview. Long-press cancels.
- ☐ Audio cues: drop, win, draw — reusing the existing `sdk.playSound()` pattern with new sample IDs.
- ☐ Theming: Red/Yellow color scheme conforms to platform tokens (`packages/sdk` theme exports).
- ☐ Spectate: rendered-table mode works; piece drops animate for spectators.
- ☐ Replay: replays of C4 games render via the same replay infrastructure as TTT.
- ☐ Bot vs human: server-side bot moves work for built-in Easy/Medium/Hard. Master is not yet wired here (Phase B5).
- ☐ Tests: e2e casual match vs each built-in tier on desktop + mobile viewports.

B3 — C4 ranked + tournament

Goal: Ranked best-of-2 and tournament best-of-3 work for C4 with zero new infrastructure — A2's match layer just lights up.

- ☐ Wire C4 into ranked match orchestration. Alternating colors, ELO updates per match.
- ☐ Wire C4 into the tournament service. Best-of-3 with random-color game 3 tiebreaker.
- ☐ Classification ladder: C4 ELO tracked separately from TTT (already supported by `GameElo (userId, gameId)` unique constraint).

- ☐ Landing nav: C4 appears in the games picker; BotFilterBar shows both games.
- ☐ Tests: ranked best-of-2 happy path, tournament with 1-1 game-3 tiebreaker, classification ladder shows independent TTT and C4 ratings for the same user.

B4 — C4 training, all five algorithms

Goal: Every algorithm trains through the A3 architecture with no platform-side changes.

- ☐ Rule-Based: heuristic rules (center preference, win-on-move, block-on-move, fork detection, edge play). Zero-time "training."
- ☐ Minimax (Quick Bots): tier-label bump only, same pattern as TTT Quick Bots — `user:<id>:minimax:<tier>` for C4.
- ☐ MCTS: classical UCB1 with rollouts. Tunes simulation count, exploration constant, optional rollout policy.
- ☐ DQN: experience replay + target network. Standard preset hyperparams.
- ☐ AlphaZero: policy/value network with PUCT-guided MCTS. Three presets:
  - Quick — ~30 minutes, small network, low simulation count. Auto-confirmed.
  - Standard — ~4 hours, medium network. **Hits admin-approval threshold.**
  - Deep — ~40 hours, full network + high sim count. Always admin-approved.
- ☐ Eval curves: primary opponent per algorithm (e.g., AZ evals against Hard minimax). Easy + Medium side curves render saturated and auto-fade.
- ☐ Streaming UX: TTT users seeing W/D/L curves in realtime today will see the same UI for C4 with no relearning.
- ☐ Tests: each algorithm reaches expected milestones from a known seed in CI-bounded episode counts.

B5 — Master tier + Master Challenge

Goal: Perfect-play Master, off-ladder, with the Master Challenge user flow.

- ☐ Master button on the C4 home: opens a best-of-2 Master Challenge (one game as first-mover, one as Master first-mover).
- ☐ Master plays the perfect-play solver from B1. Move latency budgeted (solver is fast enough at depth-required for solved-game optimality, but cache opening lines).
- ☐ No ELO update on Master matches. Per-bot "vs Master" stat block on the bot detail page (wins / draws / losses).
- ☐ Training integration: AlphaZero bots can opt into "vs Master" eval as an additional curve (showing first-mover-draw rate over time — the AlphaZero teaching moment).
- ☐ Landing UI: Master section on C4 home page explains the off-ladder rule and what "beating Master" means (linking to `Help_Corpus/solved-games-and-master.md`).
- ☐ Tests: Master Challenge happy path, off-ladder ELO behavior (no rating movement), AZ training with Master-vs-bot eval curve.

Phase B exit criteria

- ☐ C4 ships through `/games/connect-four/...` for casual, ranked, tournament, and Master Challenge.
- ☐ All five algorithms train through the A3/A3b architecture. AZ Deep on C4 succeeds at least once on staging without affecting API latency.
- ☐ Multi-skill bots can hold both a TTT and a C4 skill, each with independent ELO.
- ☐ Replay, spectate, ranked classification all work for C4 at parity with TTT.
- ☐ Corpus matches reality — the Connect 4 pages in `Help_Corpus/` reflect what shipped.

Phase C — Polish

Approximately 3-4 sprints. Lower urgency than Phase B but completes the end-to-end experience.

C1 — Journey + coaching updates

- ☐ Update the onboarding journey to surface a Connect 4 milestone after the user's first ranked TTT win.
- ☐ Add coaching cards for C4 specifics (column thinking, first-mover advantage, threat parity).
- ☐ Update `Intelligent_Guide_Requirements.md` and `Intelligent_Guide_Implementation_Plan.md`.

C2 — Training UX completions

- ☐ Stop-early button on active sessions (with confirmation + "save current checkpoint").
- ☐ 7-day rollback button on bot detail page: revert a skill's weights to a previous checkpoint within a 7-day retention window.
- ☐ Advanced disclosure (v1.1): expose hyperparameter knobs behind an "Advanced" toggle for users who want to tune past presets.

C3 — Corpus completions

- ☐ Write the queued cost/preset docs: "What each preset does", "How long does training take?", "What does training cost?", "What happens if I close the tab?", "Stop training early", "Why no knob tweaking", "Why AZ slow on C4 fast on TTT".
- ☐ Refresh `bots-overview.md`, `gym-train-tab.md`, `understanding-training-charts.md` with C4-specific examples.
- ☐ Render PDFs via the tuned pandoc invocation.

C4 — Accessibility + mobile polish

- ☐ Keyboard navigation for the C4 board (arrow keys + enter to drop).
- ☐ Screen-reader labels for board state ("Column 4, row 3, your turn").
- ☐ Mobile UX refinement pass — confirm the tap-to-preview-tap-to-confirm gesture works cleanly across viewports and doesn't conflict with browser swipe gestures.

C5 — Observability sweep

- ☐ C4-specific dashboards: completion rate per tier, AZ Master-draw-rate over time, training queue depth, worker CPU/RAM under typical load.
- ☐ Alert thresholds: dead-letter queue depth, training session failures per hour, API latency regression during heavy training.

Risks + mitigations

Risk	Mitigation
Slug-rename migration corrupts production data. Renaming `xo` → `tic-tac-toe` across `BotSkill`, `GameElo`, `Tournament` is a write-heavy migration.	Dry-run on staging with a prod data snapshot. Migration runs inside a transaction with explicit row counts logged. Rollback plan: a reverse-rename migration is staged before deploy.
Match-level ELO produces unexpected user-visible drops or jumps at cutover. Even with frozen historical rows, the first few matches under the new formula will move ratings under different math.	Communicate proactively in release notes. The new math gives smaller per-match deltas (it's "match score 0.5" vs "game-loss 0.0"), so the practical effect is gentler movement. Monitor for outliers.
AlphaZero Deep run on C4 starves the API box despite the worker. Per-user cap and worker isolation should prevent this, but a misconfigured Fly app could fall back to in-process.	Pre-launch load test on staging: queue a real AZ Deep run while a separate user plays ranked. API p95 latency must not regress.
SDK gap surfaces mid-Phase B. Discovering during B2 that the SDK can't express column-input semantics would block the sprint.	Phase A1 audits explicitly for this. If a gap appears in B, the SDK is extended in place; we don't ship workarounds.
Master solver is too slow at depth-required. Perfect-play C4 needs deep search; if move latency is too high, the Master tier feels broken.	Cache the entire opening book (first 10-12 plies) as a lookup table. Worst case, fall back to a precomputed move table for the opening, switch to live search after the book exits.
Mobile column-input gesture conflicts with browser scroll. A column tap shouldn't scroll the page.	`touch-action: manipulation` on the board; e2e tests on mobile viewports.
Worker deployment introduces a new failure surface. A worker outage could halt all training.	Status page reflects worker health. If the worker is down, new training-start requests return a clear "training temporarily unavailable" error and queue UI surfaces the outage. Existing ranked/casual/tournament play is unaffected (those don't go through the worker).

Sequencing summary

```
Phase A (TTT only):
  A1 — SDK + routing + slug rename
  A2 — Match-based play + match-level ELO
  A3a — Training data + UX rework (in-process)
  A3b — Worker process cutover
  A4 — Multi-skill bot UI + auto-clone

Phase B (Connect 4 build):
  B1 — packages/game-connect-four/
  B2 — C4 play surface (desktop + mobile)
  B3 — C4 ranked + tournament
  B4 — C4 training, all five algorithms
  B5 — Master tier + Master Challenge

Phase C (polish):
  C1 — Journey + coaching
  C2 — Training UX completions
  C3 — Corpus completions
  C4 — Accessibility + mobile polish
  C5 — Observability sweep
```

Phases ship sequentially. Sprints within a phase can run in parallel where they don't have ordering dependencies (e.g., B2 and B3 can overlap; B4 needs the C4 package from B1; B5 needs the C4 surface from B2).

Open items (will be answered as the plan executes)

- **Worker pool sizing.** Start with `count=1` on staging, `count=2` on prod, scale based on observed queue depth. Final sizing TBD post-launch.
- **AZ C4 Deep economics.** Per the corpus, AZ C4 Deep is roughly $3–50 per run depending on Fly machine class. The exact admin-approval UX (credits charged? burned against an admin pool?) is open — pin down before Phase B4.
- **Master solver move-cache size.** Opening-book lookup table size depends on how deep the book runs. Spike during B1.
- **Color naming.** Red/Yellow is canonical; settle whether colorblind-mode swaps to a high-contrast pair or uses pattern fills. Decide during B2.